

PROJET ECOTRACK

MODÉLISATION

MODÉLISATION UML

Master 1 — Filière Développement Logiciel (EADL, RNCP 38822) — Compétence C2.1

MODÉLISATION UML

Cette section présente la modélisation UML de la plateforme ECOTRACK pour le périmètre applicatif de la filière Développement : diagramme de cas d'utilisation, diagramme de classes et diagrammes de séquence. Les diagrammes graphiques sont fournis en annexe ; chaque figure est ici accompagnée d'une description détaillée. La modélisation s'appuie directement sur les scénarios du cahier des charges (section IV) et couvre les cinq acteurs du système : Citoyen, Agent de collecte, Gestionnaire, Administrateur et le Système IoT (capteurs des conteneurs).

A. Diagramme de Cas d'Utilisation

Figure 1 — Diagramme de cas d'utilisation ECOTRACK (voir Annexe)

Le système ECOTRACK met en scène cinq acteurs. Quatre sont des acteurs humains, le cinquième est un acteur système non humain :

- Citoyen (15 000 utilisateurs actifs) — acteur principal de la dimension participative et de la gamification, via l'application mobile citoyen.
- Agent de collecte (50) — opérateur terrain, via l'application mobile agent.
- Gestionnaire (10) — supervise l'exploitation et planifie les tournées via le dashboard web.
- Administrateur (3) — administre la plateforme (utilisateurs, paramétrage, gamification) via le dashboard web.
- Système IoT — acteur secondaire représentant les capteurs des 2 000 conteneurs, qui alimentent la plateforme en mesures de remplissage.

Cas d'utilisation du Citoyen :

- UC-C01 — Signaler un conteneur plein : depuis la carte, le citoyen localise un conteneur, choisit le type de problème (« conteneur plein »), ajoute une photo optionnelle et valide. Le signalement est horodaté et géolocalisé ; le citoyen reçoit +10 points de gamification (relation « include » vers Crédit de points) et une notification part vers le gestionnaire de zone. Exception : doublon (< 1 h), conteneur inexistant.
- UC-C02 — Consulter son historique et son impact : statistiques personnelles (signalements, points, badges), impact environnemental (CO₂ économisé) et position dans le classement.
- UC-C03 — Participer à un défi collectif : le citoyen s'inscrit à un défi (ex. « Zéro débordement — Quartier Centre »), effectue des signalements pendant la période, puis les résultats et récompenses sont calculés à la clôture.

Cas d'utilisation de l'Agent de collecte :

- UC-A01 — Recevoir la tournée du jour : après authentification mobile, l'agent visualise sa tournée optimisée (itinéraire, étapes numérotées, nombre de conteneurs, distance, durée estimée) puis démarre la tournée, activant le tracking.

- UC-A02 — Valider une collecte : à chaque arrêt, l'agent scanne le QR code du conteneur (ou sélection manuelle), confirme la collecte avec le volume ; le système enregistre timestamp, agent, volume et position GPS, puis affiche l'étape suivante.
- UC-A03 — Signaler une anomalie : l'agent déclare une anomalie terrain (conteneur endommagé, accès bloqué), choisit le type, ajoute commentaire et/ou photo ; le système enregistre et alerte le gestionnaire.

Cas d'utilisation du Gestionnaire :

- UC-G01 — Créer une tournée optimisée : le gestionnaire sélectionne une zone et un seuil de remplissage (ex. > 70 %) ; le système propose les conteneurs éligibles puis lance l'optimisation algorithmique (TSP avec amélioration 2-opt) qui calcule l'itinéraire optimal. Le gestionnaire ajuste si besoin, valide et assigne la tournée à un agent (« include » UC-A01 côté agent).
- UC-G02 — Monitoring temps réel : dashboard cartographique où les conteneurs sont colorés par état, identification des conteneurs critiques (> 90 %), réception des alertes en temps réel et déclenchement éventuel d'une collecte d'urgence.
- UC-G03 — Générer un rapport mensuel : sélection d'une période et de KPIs (collectes, distances, débordements), génération d'un rapport PDF structuré avec graphiques, téléchargement ou envoi par email.

Cas d'utilisation de l'Administrateur :

- UC-AD01 — Gérer les utilisateurs : depuis le backoffice, l'administrateur consulte, crée, modifie ou désactive des comptes et assigne rôles et permissions (RBAC). Toutes les actions sont tracées dans les logs d'audit.
- UC-AD02 — Configurer les alertes : définition des seuils d'alerte par type de conteneur, des destinataires des notifications et activation/désactivation des canaux (email, SMS, push).

Cas d'utilisation transversaux :

- UC-T01 — Authentification sécurisée : tout acteur humain s'authentifie par email et mot de passe ; pour le Gestionnaire et l'Administrateur, une double authentification (MFA / code TOTP) est exigée. En cas de succès, un JWT à durée limitée est émis. Blocage du compte après 5 tentatives échouées. Ce cas est en relation « include » avec l'ensemble des cas des acteurs humains.
- UC-T02 — Réception de données IoT : porté par l'acteur Système IoT. Chaque capteur mesure le niveau de remplissage (ultrason) et publie via MQTT (container_id, fill_level, temperature, timestamp), toutes les 15 minutes ou sur changement significatif. Le broker authentifie le message, les données transitent par Kafka puis PostgreSQL, et le dashboard se met à jour en temps réel. Ce cas « extend » la mise à jour de la carte et le déclenchement automatique d'alertes lorsque les seuils sont franchis.

B. Diagramme de Classes

Figure 2 — Diagramme de classes ECOTRACK (voir Annexe)

Le diagramme de classes décrit le domaine métier d'ECOTRACK. On distingue le noyau exploitation (Utilisateur, Conteneur, Zone, Mesure, Tournée, Signalement, Alerte) et le module central de gamification (Point, Badge, Défi, Classement).

Cœur du domaine :

- Utilisateur — attributs : id, nom, prenom, email, motDePasseHash, role, secretMFA, mfaActive, dateCreation, statut. Méthodes : sAuthentifier(), verifierMFA(), changerMotDePasse(). La classe Utilisateur est spécialisée (héritage) en quatre sous-types selon le rôle : Citoyen, Agent, Gestionnaire, Administrateur — chacun portant ses comportements propres (ex. Citoyen.totalPoints(), Agent.tourneeDuJour(), Gestionnaire.creerTournee(), Administrateur.gererUtilisateurs()).
- Conteneur — attributs : id, codeQR, type, capacite, latitude, longitude, niveauActuel, statut. Méthodes : estPlein(), estCritique(), derniereMesure(), historiqueMesures(). Le Conteneur appartient à une Zone et est suivi par le Système IoT.
- Zone — attributs : id, nom, secteur, geometrie (polygone PostGIS), gestionnaireResponsable. Méthodes : conteneurs(), tauxRemplissageMoyen(). Découpage des 12 secteurs géographiques.
- Mesure — attributs : id, dateHeure, niveau, temperature, qualiteSignal, source. Méthode : estAuDessusSeuil(seuil). Donnée unitaire remontée par le capteur d'un Conteneur.
- Tournée — attributs : id, date, statut, distanceTotale, dureeEstimee, seuilDeclenchement. Méthodes : optimiser() (TSP / 2-opt), ajouterArret(), genererFeuilleDeRoute(), demarrer(), cloturer(). Résultat de l'optimisation algorithmique, assignée à un Agent. La séquence des arrêts est portée par une classe d'association Arret (ordre, heurePrevue, heureReelle, volumeCollecte, statutCollecte).
- Signalement — attributs : id, type (conteneurPlein, anomalie...), dateHeure, latitude, longitude, photoUrl, commentaire, statut. Méthodes : enregistrer(), estDoublon(). Émis par un Citoyen (UC-C01) ou un Agent (UC-A03) à propos d'un Conteneur.
- Alerte — attributs : id, type, niveauGravite, dateCreation, statut, canalNotification. Méthodes : declencher(), acquitter(), escalader(). Naît du dépassement d'un seuil sur un Conteneur (UC-T02) ou d'un Signalement, et notifie le Gestionnaire (UC-G02).

Module de gamification (central) :

- Point — attributs : id, valeur, motif (ex. signalement validé), dateAttribution. Méthode : crediter(). Chaque action éligible d'un Citoyen génère des Points.
- Badge — attributs : id, libelle, description, iconeUrl, conditionObtention. Méthode : estDebloque(citoyen). Plus de 30 badges récompensant la progression.
- Défi — attributs : id, titre, description, dateDebut, dateFin, objectif, recompense, statut. Méthodes : inscrire(citoyen), calculerResultats(), attribuerRecompenses(). Support des défis collectifs (UC-C03).
- Classement — attributs : id, periode, portee (globale / par zone), dateCalcul. Méthodes : calculerRang(citoyen), top(n). Leaderboard consultable par les Citoyens (UC-C02).

Relations et cardinalités :

- Utilisateur <|-- Citoyen, Agent, Gestionnaire, Administrateur (héritage / spécialisation par rôle).
 - Zone 1 -- 0..* Conteneur (une Zone regroupe plusieurs Conteneurs).
 - Conteneur 1 -- 0..* Mesure (un Conteneur produit de nombreuses Mesures dans le temps).
 - Conteneur 1 -- 0..* Alerte et Conteneur 1 -- 0..* Signalement.
 - Tournée 0..* -- 1..* Conteneur via la classe d'association Arret (un Conteneur peut figurer dans plusieurs Tournées).
 - Agent 1 -- 0..* Tournée (un Agent réalise plusieurs Tournées ; une Tournée est assignée à un Agent).
 - Citoyen 1 -- 0..* Signalement et Citoyen 1 -- 0..* Point.
 - Citoyen 0..* -- 0..* Badge (un Citoyen débloque plusieurs Badges, un Badge est partagé par plusieurs Citoyens).
 - Citoyen 0..* -- 0..* Défi (inscription aux défis) et Défi 1 -- 0..1 Classement de défi.
- Cette structure orientée objet sépare clairement les données mesurées (Mesure), les entités physiques (Conteneur, Zone), le pilotage opérationnel (Tournée, Arret, Alerte, Signalement), la gestion des accès (Utilisateur et ses rôles) et l'engagement citoyen (gamification), ce qui facilite le découpage en services et le mapping objet-relationnel vers PostgreSQL/PostGIS.

C. Diagrammes de Séquences

Figure 3 — Séquence « Signalement citoyen d'un conteneur plein » (voir Annexe)

Ce scénario (UC-C01) illustre l'enchaînement entre l'application mobile citoyen, l'API REST, le module de gamification et le mécanisme d'alerte. Participants : Citoyen, App mobile citoyen, API/Service Signalement, Service Gamification, Service Alertes, Gestionnaire.

1. Le Citoyen ouvre l'application et localise le conteneur sur la carte (GPS).
2. Il sélectionne « Signaler un problème », choisit le type « Conteneur plein » et ajoute une photo optionnelle, puis valide.
3. L'application envoie une requête POST /signalements (container_id, type, position, photo) à l'API, accompagnée du JWT.
4. Le Service Signalement vérifie l'absence de doublon récent (estDoublon(), < 1 h) et que le conteneur existe, puis crée et persiste le Signalement horodaté et géolocalisé.
5. En cas de succès, le Service Signalement appelle de façon asynchrone le Service Gamification : crediter(+10 points) au profil du Citoyen, mise à jour des badges et du classement.
6. En parallèle, le Service Signalement notifie le Service Alertes, qui rattache l'information au Conteneur concerné et notifie le Gestionnaire de zone (alerte temps réel via WebSocket / push).
7. L'API renvoie la confirmation à l'application ; le Citoyen voit son nouveau total de points et l'accusé de signalement.

Flux alternatif : si l'étape 4 détecte un doublon ou un conteneur inexistant, l'API renvoie une erreur 409 / 404, aucun point n'est crédité et aucune alerte n'est créée.

Figure 4 — Séquence « Réception d'une mesure IoT + déclenchement d'alerte » (voir Annexe)

Ce scénario (UC-T02) décrit la chaîne d'ingestion temps réel des mesures de remplissage.

Participants : Capteur (Système IoT), Broker MQTT (Mosquitto, TLS), Service d'ingestion, Kafka, Service de traitement, Base PostgreSQL/PostGIS, Dashboard (carte temps réel), Service Alertes, Gestionnaire.

1. Le Capteur d'un Conteneur mesure le niveau (ultrason) toutes les 15 minutes ou sur changement significatif, et publie un message MQTT (container_id, fill_level, temperature, timestamp).
2. Le Broker MQTT authentifie le capteur (certificat TLS) et relaie le message.
3. Le Service d'ingestion consomme le message, le valide et le publie dans un topic Kafka (découplage et absorption de la charge, ~400 messages/minute).
4. Le Service de traitement consomme l'événement Kafka, crée un objet Mesure et le persiste dans PostgreSQL/PostGIS, associé au Conteneur.
5. Le service met à jour le niveauActuel du Conteneur et pousse la mise à jour vers le Dashboard (WebSocket) : le marqueur du conteneur change de couleur sur la carte temps réel.
6. Le service évalue le seuil via estAuDessusSeuil() / estCritique(). Si le seuil configuré (UC-AD02, ex. > 90 %) est franchi, il invoque le Service Alertes : declencher() crée une Alerte rattachée au Conteneur.
7. Le Service Alertes notifie le Gestionnaire en temps réel via le canal configuré (push / email / SMS). Le Gestionnaire peut alors acquitter() l'alerte ou déclencher une collecte d'urgence (UC-G02).

Flux alternatif : si le seuil n'est pas atteint, le flux s'arrête après la mise à jour de la carte (étape 5), sans création d'alerte.

CONCLUSION

Le choix d'UML s'impose pour ECOTRACK côté Développement car la plateforme est avant tout une application orientée objet, structurée en services (authentification, ingestion IoT, optimisation des tournées, gamification, gestion des alertes, cartographie) et déclinée en trois clients : application mobile citoyen, application mobile agent et dashboard web. UML permet de décrire simultanément le périmètre fonctionnel par acteur (cas d'utilisation), la structure statique du domaine (diagramme de classes, incluant le module de gamification) et le comportement dynamique inter-services (diagrammes de séquence), couvrant ainsi l'ensemble des besoins de conception logicielle du projet. BPMN aurait été pertinent pour décrire des processus métier organisationnels transverses, mais il ne modélise ni les objets, ni leurs attributs et méthodes, ni les contrats d'interface entre services : il reste trop centré « flux » pour guider l'implémentation d'un système objet et de ses API REST. MERISE se concentre, lui, sur le modèle de données relationnel (MCD / MLD / MPD) ; or ECOTRACK manipule des données hétérogènes — séries temporelles IoT, données géospatiales PostGIS, files de messages MQTT/Kafka, données de gamification — qui ne se réduisent pas à un schéma purement relationnel, et MERISE n'exprime ni les comportements ni les interactions entre composants. UML offre donc la meilleure correspondance avec une architecture orientée objet et orientée services : les classes se traduisent directement en entités du domaine et en modèles de données, les diagrammes de séquence documentent les échanges entre composants (MQTT, Kafka, API REST, WebSocket) et le diagramme de cas d'utilisation cadre le périmètre fonctionnel des cinq acteurs. Cette cohérence entre modèle et code accélère la conception, fiabilise les développements et facilite la maintenance évolutive de la plateforme.